

Assignment 1 – Phase 2

Pond Catchment Analysis Backend

AI-based Village Pond Planning System

Student: Aayush Kumar

Contents

1	Project Overview	2
2	GitHub Repository	2
3	Working API Route	2
4	Input File Format	3
5	Catchment Estimation Approach	3
5.1	Step 1: File Validation and Upload	3
5.2	Step 2: Contour Extraction	3
5.3	Step 3: Terrain Reconstruction	4
5.4	Step 4: Digital Elevation Model	4
5.5	Step 5: Coordinate Reference System	5
5.6	Step 6: Slope Analysis	5
5.7	Step 7: Pond Candidate Identification	6
5.8	Step 8: Flow Direction	7
5.9	Step 9: Flow Accumulation	7
5.10	Step 10: Catchment Delineation	7
5.11	Step 11: Catchment Area Calculation	8
6	Final Processing Pipeline	8
7	API Documentation	9
7.1	Endpoint	9
7.2	Description	9
7.3	Request	9
7.4	Expected Response	9
7.5	Error Handling	10
8	Demonstration Using the Provided Contour Map	10
9	Generalization and Extensibility	11
10	Limitations	12
11	Conclusion	12

1 Project Overview

The objective of Phase 2 is to develop a backend API capable of accepting a contour map in KML/KMZ format, analyzing the terrain represented by the contour data, identifying a suitable preliminary pond location, estimating the corresponding catchment area, and returning the results in a structured format.

The implementation was designed as a generalized geospatial processing pipeline rather than as a solution specific to the provided sample contour map. No pond location, coordinate, catchment boundary, or final result is hard-coded. Instead, the output is derived dynamically from the uploaded contour data.

The overall processing pipeline is:

KML/KMZ Input → Contour Parsing → Terrain Reconstruction → DEM → Slope Analysis → Pond Candidate Identification → Flow Direction → Flow Accumulation → Catchment Delineation → JSON/GeoJSON Output

2 GitHub Repository

The source code for the project is maintained in a GitHub repository.

- **GitHub Repo:** https://github.com/aayush2789/pond_catchment_backend
- **Repository:** The Assignment 1 Phase 2 backend implementation is maintained under the GitHub account above.

The repository contains the FastAPI backend, geospatial processing modules, test cases, sample input data, dependency configuration, and project documentation.

3 Working API Route

The backend exposes the following API endpoint:

POST /analyzeContour

The currently accessible API route is:

<http://10.1.75.53:5205/analyzeContour>

The endpoint accepts a contour map as an uploaded KML or KMZ file using multipart form-data.

The API can be tested by sending a request containing the contour file:

Content-Type: multipart/form-data

file = <contour_map.kml or contour_map.kmz>

If the backend is running with FastAPI, its interactive API documentation can be accessed through:

<http://10.1.75.53:5205/docs>

The Swagger interface provides an interactive way to upload the contour file and execute the endpoint.

4 Input File Format

The input to the system is a contour map provided in either KML or KMZ format.

KML (Keyhole Markup Language) stores geographic information such as points, lines, and polygons. In this project, the contour map contains contour geometries together with elevation information.

KMZ is the compressed form of KML. Therefore, the backend handles the two formats as follows:

KML file → Parse directly
KMZ file → Extract KML → Parse KML

The parser does not depend on a fixed number of contours, fixed geographic coordinates, or the filename of the sample map.

5 Catchment Estimation Approach

The catchment estimation process consists of several stages. The objective is to transform the uploaded contour information into a representation of terrain and subsequently determine the region from which surface water would flow toward a selected pond location.

5.1 Step 1: File Validation and Upload

The API first receives the uploaded file and validates its format. Only KML and KMZ files are accepted.

The file-processing workflow is:

```
Uploaded file
  |
  v
Check file type
  |
  +---- KML ----> Parse
  |
  +---- KMZ ----> Extract KML ----> Parse
```

Malformed files, unsupported file types, and missing input files are rejected with appropriate error responses.

5.2 Step 2: Contour Extraction

The KML/KMZ parser extracts the contour geometries and their corresponding elevation information.

Conceptually, each contour is represented as:

```
Contour:
  geometry = LineString
  elevation = value in metres
```

The parser converts the original KML structure into a normalized internal representation so that the subsequent terrain-analysis modules do not depend on the original XML structure.

For example:

```
[
  {
    "geometry": <contour geometry>,
    "elevation": 400
  },
  {
    "geometry": <contour geometry>,
    "elevation": 410
  }
]
```

The minimum and maximum elevations and the geographic extent of the input are also determined dynamically.

5.3 Step 3: Terrain Reconstruction

Contour maps provide elevation information only along contour lines. To perform hydrological analysis, the system reconstructs a continuous representation of the terrain between the contour lines.

The known contour information can be represented as spatial elevation samples:

$$(x_i, y_i, z_i)$$

where x_i and y_i represent the spatial location and z_i represents the known elevation.

Interpolation is then used to estimate elevation values between the known contour lines.

The conceptual transformation is:

Contour Lines $\rightarrow$ Elevation Samples $\rightarrow$ Interpolation $\rightarrow$ Terrain Elevation
Surface

The resulting elevation surface forms the basis of the Digital Elevation Model (DEM).

5.4 Step 4: Digital Elevation Model

The reconstructed terrain is represented as an elevation grid in which each cell contains an estimated terrain elevation.

A simplified representation is:

```
400  402  405  408
402  405  409  413
405  409  414  419
408  413  419  425
```

Each cell represents a portion of the terrain.

The DEM stores metadata such as:

- Minimum elevation
- Maximum elevation
- Spatial extent
- Number of rows and columns
- Grid resolution
- Coordinate reference system

The spatial extent and terrain representation are derived from the uploaded file rather than being hard-coded for the sample map.

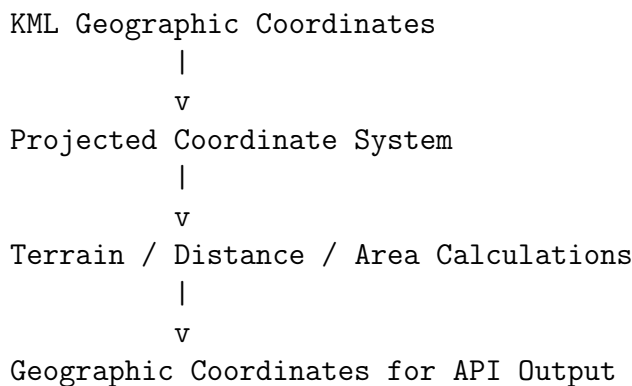
5.5 Step 5: Coordinate Reference System

The input KML uses geographic coordinates, which are typically represented using latitude and longitude.

However, distances and areas should not be calculated directly using latitude and longitude because those values are angular measurements rather than metres.

Therefore, the terrain-processing workflow uses an appropriate projected coordinate reference system when metric calculations are required.

The general process is:



This allows the catchment area to be calculated in meaningful metric units such as square metres and hectares.

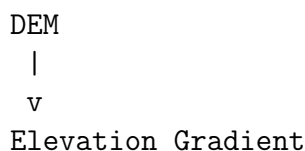
5.6 Step 6: Slope Analysis

Once the DEM has been generated, the terrain slope is calculated.

Slope represents the rate of change of elevation across the terrain.

The slope layer is used to identify regions that are relatively gentle compared with steep terrain.

Conceptually:



```

|
v
Slope Calculation
|
v
Slope Map

```

The resulting slope information is later used as one of the factors in preliminary pond-site identification.

5.7 Step 7: Pond Candidate Identification

The system identifies one or more preliminary pond candidate locations from the terrain rather than choosing a fixed coordinate.

Candidate generation considers terrain characteristics such as:

- Elevation suitability
- Terrain slope
- Local terrain configuration

A configurable scoring mechanism is used conceptually as:

$$S = w_e E + w_s S_l$$

where:

- E represents elevation suitability,
- S_l represents slope suitability,
- w_e and w_s are configurable weights.

The scoring logic is kept separate from the terrain-processing code. This design allows future information such as flow accumulation, rainfall, runoff, land availability, and soil characteristics to be incorporated without redesigning the complete system.

The candidate output contains information such as:

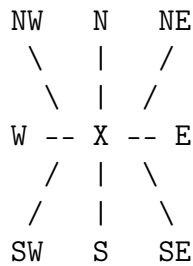
- Latitude
- Longitude
- Elevation
- Slope
- Suitability score

5.8 Step 8: Flow Direction

After a candidate location has been identified, the terrain is analyzed to determine the direction in which surface water would naturally flow.

A raster-based flow-direction approach such as the D8 method can be used. Under this approach, the flow direction of a cell is determined by examining neighbouring cells and selecting an appropriate downslope direction.

Conceptually:



This produces a flow-direction representation for the DEM.

5.9 Step 9: Flow Accumulation

Flow accumulation determines how much upstream terrain contributes flow to each DEM cell.

Cells receiving flow from many upstream cells have high flow accumulation values.

The conceptual sequence is:

DEM → Flow Direction → Flow Accumulation

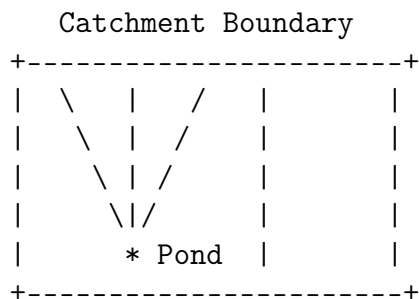
High-flow-accumulation regions represent natural drainage pathways and are useful for identifying locations where surface runoff converges.

5.10 Step 10: Catchment Delineation

The selected pond candidate is treated as an outlet or pour point.

The system traces the upstream cells that ultimately drain toward this location.

Conceptually:



All cells contributing surface flow to the outlet form the estimated catchment.

The catchment therefore represents:

All upstream terrain contributing runoff to the selected pond location.

5.11 Step 11: Catchment Area Calculation

After the contributing cells are identified, the catchment area is calculated.

If each raster cell has an area A_c and the catchment contains N cells, a simplified raster area calculation is:

$$A_{\text{catchment}} = N \times A_c$$

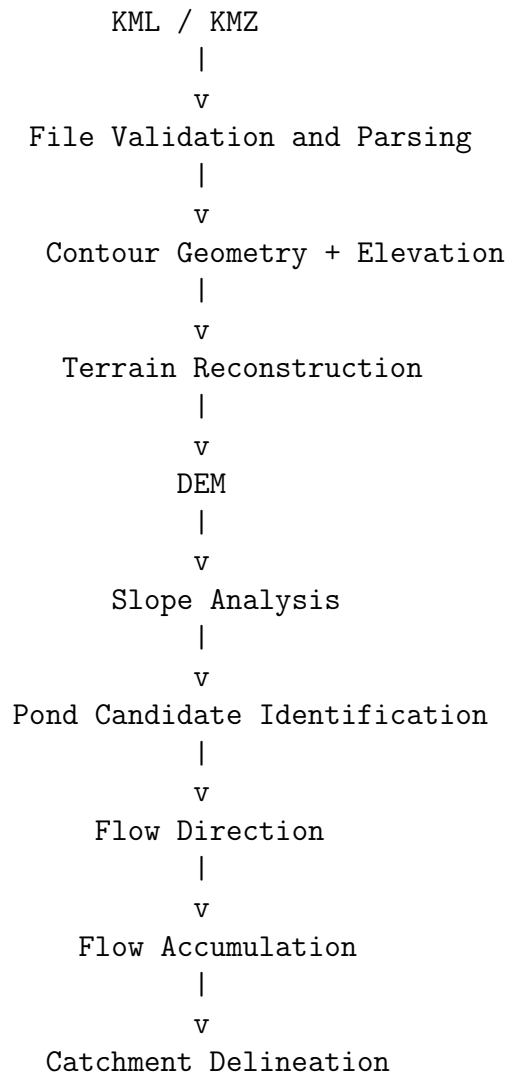
The result is reported in:

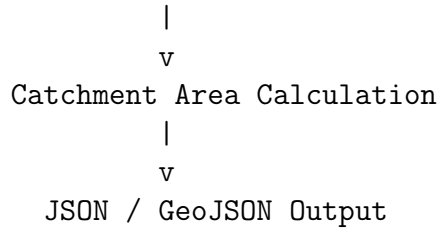
- Square metres (m^2)
- Hectares (ha)

The area is calculated using metric/projected coordinates rather than directly from latitude and longitude.

6 Final Processing Pipeline

The complete backend processing flow is:





7 API Documentation

7.1 Endpoint

POST /analyzeContour

7.2 Description

The endpoint accepts a contour map in KML or KMZ format and performs terrain and catchment analysis. It returns a preliminary pond candidate together with the corresponding catchment information.

7.3 Request

The request is sent using multipart form-data.

Content-Type: multipart/form-data

file: contour_map.kml

or:

file: contour_map.kmz

7.4 Expected Response

A successful response is structured to contain the pond location, terrain information, and catchment information.

An example response structure is:

```
{
  "status": "success",

  "pond_site": {
    "latitude": <value>,
    "longitude": <value>,
    "elevation_m": <value>,
    "slope_deg": <value>,
    "suitability_score": <value>
  },

  "catchment": {
```

```

        "area_m2": <value>,
        "area_hectares": <value>,
        "flow_accumulation": <value>
    },

    "catchment_geometry": {
        "type": "Polygon",
        "coordinates": [...]
    }
}

```

The values returned by the actual implementation are calculated dynamically from the uploaded contour map.

7.5 Error Handling

The API handles invalid input and processing errors using structured HTTP responses. Examples include:

- Unsupported file type
- Missing uploaded file
- Malformed KML/KMZ
- Missing contour elevation information
- Invalid contour geometry
- Insufficient terrain information
- Terrain interpolation failure
- Invalid pond candidate
- Catchment delineation failure

The API returns a descriptive error message instead of exposing internal implementation details or stack traces.

8 Demonstration Using the Provided Contour Map

The provided sample contour map was used as the test input for the backend.

The demonstration workflow is:

1. Start the FastAPI backend.
2. Open the API documentation at:

<http://10.1.75.53:5205/docs>

3. Select the POST `/analyzeContour` endpoint.
4. Use the “Try it out” option.
5. Upload the provided KML/KMZ contour map.
6. Execute the request.
7. The backend parses the contour data and reconstructs the terrain.
8. Terrain slope and candidate locations are calculated.
9. Flow direction and flow accumulation are derived.
10. The upstream catchment corresponding to the selected candidate is delineated.
11. The resulting pond location and catchment information are returned as structured JSON/GeoJSON.

A successful execution demonstrates that the backend can process the provided contour map end-to-end without requiring hard-coded coordinates or precomputed sample results.

9 Generalization and Extensibility

An important requirement of the assignment is that the implementation should not be specific to the provided sample contour map.

The backend was therefore designed as a modular pipeline in which individual stages perform separate responsibilities:

```

API Layer
|
+-- File Parser
|
+-- Terrain Service
|
+-- Candidate Recommendation Service
|
+-- Flow Analysis Service
|
+-- Catchment Service
|
+-- Response Builder

```

The same architecture can be extended to future versions of the project. For example, the current pipeline ends at:

Catchment

while future versions can continue with:

Catchment → Rainfall → Runoff → Storage Capacity → Pond Dimensions

Similarly, the candidate scoring module can later incorporate land availability, soil characteristics, rainfall, runoff, and other planning parameters.

10 Limitations

The current system provides a preliminary terrain-based catchment estimation and is intended for pond-site planning at an initial screening stage.

The result should not be interpreted as a substitute for:

- Detailed topographic surveying
- Field verification
- Soil investigation
- Professional hydrological analysis
- Structural engineering design
- Legal verification of land ownership

The accuracy of the result also depends on the quality and resolution of the input contour data.

11 Conclusion

Assignment 1 Phase 2 implements the core geospatial backend required for preliminary pond catchment analysis. The system accepts KML/KMZ contour maps, extracts elevation information, reconstructs a terrain model, analyzes slope, identifies a preliminary pond candidate, determines surface flow behavior, delineates the corresponding catchment, and returns the result through a structured API.

The implementation is designed to be generalized rather than specific to the provided sample map. Its modular structure also provides a foundation for future phases involving rainfall, runoff estimation, pond storage, land suitability, and more comprehensive pond planning.